



Gobierno de
México

Ciencia y Tecnología
Secretaría de Ciencia, Humanidades, Tecnología e Innovación

UNRC
Universidad Nacional Rosario Castellanos



ESTRUCTURA DE DATOS



2025
Año de
**La Mujer
Indígena**

Avenida 506 Col. San Juan de Aragón II Sección, Gustavo A. Madero C.P. 07969, Ciudad de México www.rcastellanos.cdmx.gob.mx



Descripción del Programa

Objetivo del programa

El programa desarrollado en lenguaje C++ tiene como objetivo implementar un sistema básico de bitácora digital para el sistema PAD-Bienestar, una plataforma orientada a la gestión de atención a usuarios en servicios de bienestar institucional. El programa permite registrar de forma persistente el número de sesiones activas y de turnos atendidos durante cada jornada de operación del sistema, almacenando dichos datos en un archivo de texto denominado bitacora.txt. Adicionalmente, ofrece la posibilidad de consultar en pantalla todos los registros previamente guardados.

¿Qué es una bitácora digital?

Una bitácora digital es un registro cronológico y estructurado de eventos, actividades o datos generados por un sistema informático a lo largo del tiempo. En el ámbito de los sistemas de información, la bitácora cumple la función de documentar de manera continua las operaciones realizadas, los valores registrados y el estado del sistema en distintos momentos. A diferencia de un simple archivo de notas, la bitácora digital está diseñada para ser consultada, auditada y analizada, lo que la convierte en una herramienta fundamental para el monitoreo, la trazabilidad y la toma de decisiones dentro de cualquier organización que opere sistemas digitales.

Importancia de guardar información en archivos

En programación, la información almacenada únicamente en la memoria RAM del equipo (memoria volátil) se pierde en el instante en que el programa finaliza su ejecución. Los archivos de texto o binarios permiten que los datos generados durante una sesión persistan más allá de la vida del proceso, haciéndolos disponibles para ejecuciones futuras, reportes o auditorías (Kernighan & Ritchie, 1988).

Persistencia: los datos sobreviven al cierre del programa y pueden recuperarse en cualquier momento posterior.

Portabilidad: un archivo de texto puede ser abierto, revisado y compartido con cualquier editor sin necesidad de software especializado.

Auditoría y trazabilidad: es posible verificar qué ocurrió en el sistema en fechas y horas específicas.



Escalabilidad: el volumen de datos no está limitado por la cantidad de memoria RAM disponible.

Separación de datos y lógica: el programa y los datos pueden evolucionar de forma independiente.

Funcionamiento del Programa

Información que guarda el programa

Cada vez que el usuario selecciona la opción de registro, el programa captura y almacena en el archivo bitacora.txt los siguientes tres datos:

Campo	Tipo	Descripción
Fecha y hora	Automático	Timestamp del sistema al momento del registro
Sesiones activas	Entero positivo	Número de sesiones abiertas en el día
Turnos atendidos	Entero positivo	Número de turnos procesados durante la jornada



Cómo se guardan los datos en el archivo

El proceso de escritura utiliza la función `fopen()` con el modo de apertura "a" (append), lo que garantiza que cada nuevo registro se añade al final del archivo sin sobrescribir los datos anteriores. Posteriormente, la función `fprintf()` escribe los datos con formato legible, y finalmente `fclose()` cierra el archivo para asegurar que los cambios queden guardados en disco. El fragmento central del proceso de escritura es el siguiente:

```
archivo = fopen(NOMBRE_ARCHIVO, "a");
if (archivo == NULL)
{
    printf("\n [ERROR] No se pudo abrir el archivo '%s'.\n", NOMBRE_ARCHIVO);
    return;
}

fprintf(archivo, "-----\n");
fprintf(archivo, "Fecha y hora      : %s\n", fechaHora);
fprintf(archivo, "Sesiones activas : %d\n", sesiones);
fprintf(archivo, "Turnos atendidos : %d\n", turnos);
fprintf(archivo, "-----\n\n");

fclose(archivo);

printf("\n [OK] Registro guardado correctamente en '%s'.\n", NOMBRE_ARCHIVO);
}
```



Cómo se consulta la información almacenada

Para la consulta, el programa abre el archivo bitacora.txt en modo lectura ("r") mediante fopen() y recorre todas las líneas con un bucle usando fgets(), que lee una línea completa por iteración hasta encontrar el final del archivo (EOF). Cada línea leída se imprime directamente en pantalla. Al terminar, fclose() cierra el archivo de forma segura:

```
archivo = fopen(NOMBRE_ARCHIVO, "r");
if (archivo == NULL)
{
    printf("\n [INFO] No existe ningún registro aún en '%s'.\n", NOMBRE_ARCHIVO);
    printf("      Realice primero un registro de datos.\n");
    return;
}

printf("\n Contenido del archivo '%s':\n\n", NOMBRE_ARCHIVO);

while (fgets(linea, BUFFER_SIZE, archivo) != NULL)
{
    printf("  %s", linea);
    /* Contar secciones de registro (cada separador = 1 registro) */
    if (strstr(linea, "Sesiones activas") != NULL)
    {
        registros++;
    }
}

fclose(archivo);
```

Archivo vs. memoria RAM: ¿por qué usar archivos?

Mantener la información únicamente en variables de programa (memoria RAM) significa que todos los datos desaparecen cuando el proceso termina, sin posibilidad de recuperarlos. El uso de archivos soluciona este problema porque el sistema operativo almacena los datos de forma no volátil en el disco duro o unidad de almacenamiento, garantizando que sigan disponibles en futuras ejecuciones del programa, en otras sesiones de trabajo e incluso en otro equipo si el archivo se comparte (Stroustrup, 2013).



Código Fuente del Programa

A continuación, se presenta el código fuente completo del programa.

Encabezado y librerías

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define NOMBRE_ARCHIVO "bitacora.txt"
#define BUFFER_SIZE 256
```





Validación de entrada (validarEnteroPositivo)

```
int validarEnteroPositivo(const char *mensaje)
{
    int valor;
    int valido = 0;

    while (!valido)
    {
        printf("%s", mensaje);
        if (scanf("%d", &valor) != 1)
        {
            while (getchar() != '\n')
                ;
            printf(" [ERROR] Por favor ingrese un número entero válido.\n");
        }
        else if (valor <= 0)
        {
            printf(" [ERROR] El valor debe ser un número positivo mayor a cero.\n");
        }
        else
        {
            valido = 1;
        }
    }
    return valor;
}
```





Registro en archivo (registrarBitacora)

```
void registrarBitacora()
{
    FILE *archivo;
    char fechaHora[BUFFER_SIZE];
    int sesiones, turnos;

    printf("\n===== \n");
    printf("    REGISTRO DE USO - MARVIN - MAYRA \n");
    printf("===== \n");

    sesiones = validarEnteroPositivo(" Número de sesiones activas del día : ");
    turnos = validarEnteroPositivo(" Número de turnos atendidos      : ");

    obtenerFechaHora(fechaHora, BUFFER_SIZE);

    archivo = fopen(NOMBRE_ARCHIVO, "a");
    if (archivo == NULL)
    {
        printf("\n [ERROR] No se pudo abrir el archivo '%s'. \n", NOMBRE_ARCHIVO);
        return;
    }

    fprintf(archivo, "----- \n");
    fprintf(archivo, "Fecha y hora      : %s \n", fechaHora);
    fprintf(archivo, "Sesiones activas  : %d \n", sesiones);
    fprintf(archivo, "Turnos atendidos  : %d \n", turnos);
    fprintf(archivo, "----- \n \n");

    fclose(archivo);

    printf("\n [OK] Registro guardado correctamente en '%s'. \n", NOMBRE_ARCHIVO);
}
```




Consulta del archivo (consultarBitacora)

```
void consultarBitacora()
{
    FILE *archivo;
    char linea[BUFFER_SIZE];
    int registros = 0;

    printf("\n=====\\n");
    printf("    CONSULTA DE BITÁCORA - MARVIN - MAYRA    \\n");
    printf("=====\\n");

    archivo = fopen(NOMBRE_ARCHIVO, "r");
    if (archivo == NULL)
    {
        printf("\n [INFO] No existe ningún registro aún en '%s'.\\n", NOMBRE_ARCHIVO);
        printf("    Realice primero un registro de datos.\\n");
        return;
    }

    printf("\n Contenido del archivo '%s':\\n\\n", NOMBRE_ARCHIVO);

    while (fgets(linea, BUFFER_SIZE, archivo) != NULL)
    {
        printf("  %s", linea);
        /* Contar secciones de registro (cada separador = 1 registro) */
        if (strstr(linea, "Sesiones activas") != NULL)
        {
            registros++;
        }
    }

    fclose(archivo);
}
```





Evidencia de Ejecución

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE

Bienvenido al sistema de Bitácora - PAD-Bienestar MARVIN - MAYRA

=====
      SISTEMA PAD-Bienestar | MARVIN - MAYRA
=====
1. Registrar uso del día
2. Consultar bitácora guardada
3. Salir del programa
-----
Seleccione una opción [1-3]: 
```

Consulta de la bitácora (opción 2)

```
2. Consultar bitácora guardada
3. Salir del programa
-----
Seleccione una opción [1-3]: 2

=====
      CONSULTA DE BITÁCORA - MARVIN - MAYRA
=====

[INFO] No existe ningún registro aún en 'bitacora.txt'.
      Realice primero un registro de datos.

=====
```





Registro de uso del día (opción 1)

```
Seleccione una opción [1-3]: 2

=====
CONSULTA DE BITÁCORA - MARVIN - MAYRA
=====

Contenido del archivo 'bitacora.txt':

-----
Fecha y hora      : 27/05/2026 19:25:18
Sesiones activas : 1
Turnos atendidos : 5
-----

Total de registros encontrados: 1
=====
```

Proceso de compilación y errores resueltos

Durante el desarrollo se presentaron los siguientes inconvenientes menores y su solución:

Buffer sucio en stdin: al ingresar texto en lugar de número, el buffer de entrada quedaba con caracteres basura, causando bucles infinitos. Se resolvió agregando un ciclo while (getchar() != '\n'); para vaciar el buffer.

```
while (getchar() != '\n')
    ;
```



Conclusiones

Aprendizaje sobre manejo de archivos en C++: el desarrollo de esta práctica permitió comprender en profundidad cómo funcionan las funciones estándar de C para manejo de archivos — fopen, fprintf, fgets y fclose — y la diferencia crítica entre los modos de apertura "r", "w" y "a". Queda claro que el modo append es fundamental para sistemas de bitácora que deben acumular registros a lo largo del tiempo.

Importancia de la persistencia de datos:

esta práctica pone en evidencia que cualquier sistema de información útil en un contexto real necesita guardar sus datos de forma permanente. La memoria RAM es veloz pero efímera; los archivos y las bases de datos son la solución para que la información sobreviva más allá de la sesión de trabajo, sea auditable y pueda compartirse.

Validación de entradas como buena práctica:

implementar validación evita que el programa reciba datos incoherentes (negativos, cero o no numéricos) que podrían comprometer la integridad de la bitácora. Esta práctica de defensa en la entrada de datos es un principio básico de robustez en cualquier sistema.

Otros casos de uso de bitácoras digitales:

el concepto de bitácora es aplicable a una amplia variedad de sistemas: registros de acceso en sistemas de seguridad informática, logs de errores en servidores web, historial de transacciones en sistemas bancarios, registros de mantenimiento en equipos industriales, seguimiento de actividad en plataformas educativas en línea y monitoreo de redes de telecomunicaciones, entre muchos otros.



Gobierno de
México

Ciencia y Tecnología
Secretaría de Ciencia, Humanidades, Tecnología e Innovación

UNRC
Universidad Nacional Rosario Castellanos



Referencias:

cppreference.com. (2023). *std::fopen*. <https://en.cppreference.com/w/cpp/io/c/fopen>

cppreference.com. (2023). *std::fgets*. <https://en.cppreference.com/w/cpp/io/c/fgets>

cppreference.com. (2025). *std::fclose*. <https://en.cppreference.com/w/cpp/io/c/fclose>

Deitel, P. J., & Deitel, H. M. (2016). *C++ how to program* (10.^a ed.). Pearson Education.
<https://www.pearson.com/en-us/subject-catalog/p/Deitel-C-How-to-Program-10th-Edition/P200000009246/9780134448237>

Stroustrup, B. (2013). *The C++ programming language* (4.^a ed.). Addison-Wesley.
<https://www.stroustrup.com/4th.html>

GNU Project. (2025). *GCC online documentation*. Free Software Foundation.
<https://gcc.gnu.org/onlinedocs/>



2025
Año de
**La Mujer
Indígena**